深度之眼
deepshare.net

PINN实战演练：手把手搭建波动方程求解器

主讲人：择木老师

深度之眼

2025 年 2 月 25 日

目录

深度之眼
deepshare.net

PINN一维波动方程正问题求解

PINN一维波动方程逆问题求解

Section 1

PINN一维波动方程正问题求解

一维波动方程正问题

深度之眼 deepshare.net

方程形式：

$$\frac{\partial^2 u}{\partial t^2} - c^2 \frac{\partial^2 u}{\partial x^2} = 0$$

初始条件：

$$u(x,0) = \sin(\pi x) + \sin(2\pi x), \quad \frac{\partial u(x,0)}{\partial t} = 0$$

边界条件：

$$u(0,t) = 0, \quad u(1,t) = 0$$

解析解： 根据 d'Alembert 公式，波动方程的解析解为：

$$u(x,t) = \frac{1}{2}\left[f(x+ct) + f(x-ct)\right]$$

其中 $f(x) = \sin(\pi x) + \sin(2\pi x)$。

目标：使用PINN求解$c = 2$工况，使得PINN的解（预测值）与解析解够足接近

PINN波动方程正问题损失函数构建

深度之眼 deepshare.net

PINN 的综合损失函数：

$$\mathcal{L}(\boldsymbol{\theta}) = \lambda_r \mathcal{L}_r(\boldsymbol{\theta}) + \lambda_{i1}\mathcal{L}_{i1}(\boldsymbol{\theta}) + \lambda_{i2}\mathcal{L}_{i2}(\boldsymbol{\theta}) + \lambda_b \mathcal{L}_b(\boldsymbol{\theta})$$

各项定义为：

$$\mathcal{L}_r(\boldsymbol{\theta}) = \frac{1}{N_r}\sum_{k=1}^{N_r}\left(\frac{\partial^2 \hat{u}_{\boldsymbol{\theta}}(\boldsymbol{x}_r^k, t_r^k)}{\partial t^2} - 2^2\frac{\partial^2 \hat{u}_{\boldsymbol{\theta}}(\boldsymbol{x}_r^k, t_r^k)}{\partial x^2}\right)^2,$$

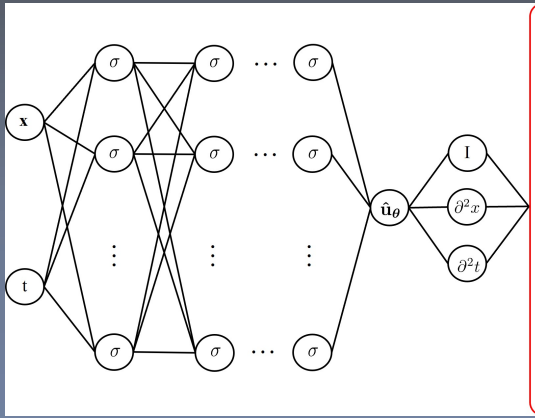
$$\mathcal{L}_{i1}(\boldsymbol{\theta}) = \frac{1}{N_i}\sum_{k=1}^{N_i}\left(\hat{u}_{\boldsymbol{\theta}}(\boldsymbol{x}_i^k, 0) - \sin(\pi x_i^k) - \sin(2\pi x_i^k)\right)^2,$$

$$\mathcal{L}_{i2}(\boldsymbol{\theta}) = \frac{1}{N_i}\sum_{k=1}^{N_i}\left(\frac{\partial \hat{u}_{\boldsymbol{\theta}}(\boldsymbol{x}_i^k, 0)}{\partial t} - 0\right)^2,$$

$$\mathcal{L}_b(\boldsymbol{\theta}) = \frac{1}{N_b}\sum_{k=1}^{N_b}\left(\hat{u}_{\boldsymbol{\theta}}(\boldsymbol{x}_b^k, t_b^k) - 0\right)^2.$$

PINN求解一维波动方程正问题框架

深度之眼
deepshare.net



最小化：

$$\boldsymbol{\theta}^{\star} = \arg\min_{\boldsymbol{\theta}} [\lambda_r \mathcal{L}_r(\boldsymbol{\theta}) + \lambda_{i1}\mathcal{L}_{i1}(\boldsymbol{\theta}) + \lambda_{i2}\mathcal{L}_{i2}(\boldsymbol{\theta}) + \lambda_b \mathcal{L}_b(\boldsymbol{\theta})]$$

各项定义为：

$$\mathcal{L}_r(\boldsymbol{\theta}) = \frac{1}{N_r}\sum_{k=1}^{N_r}\left(\frac{\partial^2 \hat{u}_{\boldsymbol{\theta}}(\boldsymbol{x}_r^k, t_r^k)}{\partial t^2} - 2^2\frac{\partial^2 \hat{u}_{\boldsymbol{\theta}}(\boldsymbol{x}_r^k, t_r^k)}{\partial x^2}\right)^2,$$

$$\mathcal{L}_{i1}(\boldsymbol{\theta}) = \frac{1}{N_i}\sum_{k=1}^{N_i}\left(\hat{u}_{\boldsymbol{\theta}}(\boldsymbol{x}_i^k, 0) - \sin(\pi x_i^k) - \sin(2\pi x_i^k)\right)^2,$$

$$\mathcal{L}_{i2}(\boldsymbol{\theta}) = \frac{1}{N_i}\sum_{k=1}^{N_i}\left(\frac{\partial \hat{u}_{\boldsymbol{\theta}}(\boldsymbol{x}_i^k, 0)}{\partial t} - 0\right)^2,$$

$$\mathcal{L}_b(\boldsymbol{\theta}) = \frac{1}{N_b}\sum_{k=1}^{N_b}\left(\hat{u}_{\boldsymbol{\theta}}(\boldsymbol{x}_b^k, t_b^k) - 0\right)^2.$$

阶段一：计算图构建

深度之眼 deepshare.net

网络架构定义： 选择神经网络结构，如全连接网络：

- 输入层：空间坐标(x, t)
- 隐藏层：3层，每层100个神经元，使用tanh激活函数
- 输出层：物理场预测值$u_\theta(x, t)$

自动微分配置： 构建计算图跟踪机制，支持高阶导数计算：

- **工具：** PyTorch (‘torch.autograd‘)。
- **步骤：**
  1. 定义输入 (x, t) 并设置 ‘requires_grad=True‘。
  2. 前向传播计算 $u_\theta(x, t)$。
  3. 使用 ‘torch.autograd.grad‘ 计算 $\frac{\partial^2 u}{\partial t^2}$ 和 $\frac{\partial^2 u}{\partial x^2}$。
- **示例代码：** $u_x = grad(u_pred, x, grad_outputs = torch.ones_like(u_pred), create_graph = True)[0]$

网络架构构建

深度之眼
deepshare.net

```python
# ## **2. 定义PINNs模型**
class PINN(nn.Module):
    def __init__(self, input_dim=2, output_dim=1, hidden_dim=50, num_hidden=3, activation='tanh'):
        """
        初始化PINN模型。

        参数:
        input_dim: 输入维度，默认2(x和t)
        output_dim: 输出维度，默认1(u)
        hidden_dim: 隐藏层神经元数量，默认50
        num_hidden: 隐藏层数量，默认3
        activation: 激活函数类型，默认'tanh'，可选'sin'
        """
        super(PINN, self).__init__()

        # 定义神经网络层
        self.layers = nn.ModuleList([nn.Linear(input_dim, hidden_dim)])
        for _ in range(num_hidden - 1):
            self.layers.append(nn.Linear(hidden_dim, hidden_dim))
        self.layers.append(nn.Linear(hidden_dim, output_dim))

        # 设置激活函数
        if activation == 'tanh':
            self.activation = torch.tanh
        elif activation == 'sin':
            self.activation = torch.sin
        else:
            raise ValueError("不支持的激活函数，仅支持 'tanh' 或 'sin'。")

    def forward(self, x, t):
        """前向传播，输入x和t，输出预测值u"""
        input_combined = torch.cat([x, t], dim=-1)  # 拼接输入 [batch_size, 2]
        out = input_combined
        for layer in self.layers[:-1]:
            out = self.activation(layer(out))
        out = self.layers[-1](out)  # 最后一层无激活函数
        return out
```

自动微分配置（PINN Class）

深度之眼
deepshare.net

```python
    def calculate_derivatives(self, x, t):
        """
        计算PDE残差所需的二阶导数。

        参数:
        x: 空间坐标
        t: 时间坐标

        返回:
        u_pred: 预测值
        u_xx: 二阶空间导数
        u_tt: 二阶时间导数
        """
        x = x.requires_grad_()
        t = t.requires_grad_()
        u_pred = self.forward(x, t)

        u_x = grad(u_pred, x, grad_outputs=torch.ones_like(u_pred),  create_graph=True)[0]
        u_xx = grad(u_x, x, grad_outputs=torch.ones_like(u_x), create_graph=True)[0]

        u_t = grad(u_pred, t, grad_outputs=torch.ones_like(u_pred), create_graph=True)[0]
        u_tt = grad(u_t, t, grad_outputs=torch.ones_like(u_t), create_graph=True)[0]

        return u_pred, u_xx, u_tt
```

阶段二：损失函数构建与数据准备

深度之眼 deepshare.net

1. **损失函数构建：** 根据正问题，构建包含PDE残差、边界条件和初始条件的损失函数：

$$\mathcal{L}(\boldsymbol{\theta}) = \lambda_r \mathcal{L}_r(\boldsymbol{\theta}) + \lambda_{i1} \mathcal{L}_{i1}(\boldsymbol{\theta}) + \lambda_{i2} \mathcal{L}_{i2}(\boldsymbol{\theta}) + \lambda_b \mathcal{L}_b(\boldsymbol{\theta})$$

   其中，各部分损失根据问题需求调整权重。

2. **训练数据生成：** 通过空间和时间采样，生成PDE解的样本点，同时准备边界条件和初始条件样本点：
   - 残差点$\{x_r^k\}_{k=1}^{N_r}$：均匀或随机采样
   - 边界点$\{x_b^k\}_{k=1}^{N_b}$：边界区域加密采样
   - 初始时刻点$\{x_0^k\}_{k=1}^{N_i}$：时域初始切片

损失函数构建（PINN Class）

深度之眼
deepshare.net

```python
    def losses(self, x, t):
        # 计算PDE残差
        u_pde, u_xx, u_tt = self.calculate_derivatives(x['pde'], t['pde'])
        residual = u_tt - c_true**2 * u_xx
        loss_pde = torch.mean(residual**2)

        # 计算初始条件损失
        x_ic = x['ic'].requires_grad_()
        t_ic = t['ic'].requires_grad_()
        u_ic = self.forward(x_ic, t_ic)
        u_t = grad(u_ic, t_ic, grad_outputs=torch.ones_like(u_ic), create_graph=True)[0]
        loss_ic1 = torch.mean((u_ic - torch.sin(np.pi * x['ic']) - torch.sin(2 * np.pi * x['ic']))**2)
        loss_ic2 = torch.mean(u_t**2)

        # 计算边界条件损失
        loss_bc = torch.mean(self.forward(x['bc'], t['bc'])**2)

        # 总损失
        total_loss = loss_pde + loss_ic1 + loss_ic2 + loss_bc

        # 初始化损失历史(如果尚未初始化)
        if not hasattr(self, 'history'):
            self.history = {
                'total_loss': [],
                'loss_pde': [],
                'loss_ic1': [],
                'loss_ic2': [],
                'loss_bc': []
            }

        # 记录损失
        self.history['total_loss'].append(total_loss.item())
        self.history['loss_pde'].append(loss_pde.item())
        self.history['loss_ic1'].append(loss_ic1.item())
        self.history['loss_ic2'].append(loss_ic2.item())
        self.history['loss_bc'].append(loss_bc.item())

        return total_loss, loss_pde, loss_ic1, loss_ic2, loss_bc
```

训练数据生成

深度之眼
deepshare.net

```python
def get_data(c, batch_size_pde, batch_size_ic, batch_size_bc):
    """
    生成训练和测试数据，用于PINNs求解波动方程。

    参数:
    c: 波速
    batch_size_pde: PDE点的批量大小
    batch_size_ic: 初始条件点的批量大小
    batch_size_bc: 边界条件点的批量大小

    返回:
    X, T: 网格化的空间和时间坐标
    x_test, t_test, u_test: 测试数据的坐标和真实解
    x_train, t_train, u_train: 训练数据的坐标和真实解(字典格式)
    U: 解析解的网格数据
    """
    # 定义空间和时间域，均匀划分101个点
    x = np.linspace(0, 1, 101)
    t = np.linspace(0, 1, 101)
    X, T = np.meshgrid(x, t)

    # 使用d'Alembert公式计算解析解
    f = lambda x: np.sin(np.pi * x) + np.sin(2 * np.pi * x)
    U = 0.5 * (f(X + c * T) + f(X - c * T))

    # 生成PDE点：从内部区域随机采样，排除边界
    x_pde = x[1:-1]  # 排除边界点 x=0 和 x=1
    t_pde = t[1:-1]  # 排除边界点 t=0 和 t=1
    X_pde, T_pde = np.meshgrid(x_pde, t_pde)
    X_pde_flat = X_pde.flatten()[:, None]
    T_pde_flat = T_pde.flatten()[:, None]
    U_pde_flat = U[1:-1, 1:-1].flatten()[:, None]

    indices_pde = np.random.choice(len(X_pde_flat), batch_size_pde, replace=False)
    x_pde_train = X_pde_flat[indices_pde]
    t_pde_train = T_pde_flat[indices_pde]
    u_pde_train = U_pde_flat[indices_pde]
```

阶段三：优化策略实施

深度之眼
deepshare.net

1. **优化器选择：** 使用Adam优化器和L-BFGS组合：
   - 前期：使用20,000轮次Adam快速下降，学习率通常设为$1e-3$
   - 后期：采用L-BFGS精细调优，进一步优化参数，最大轮次：10,000
2. **前向传播与反向传播：** 神经网络通过前向传播计算输出，并根据损失函数的残差进行反向传播优化网络参数。

Adam优化器

深度之眼
deepshare.net

```python
def train_pinn_adam(pinn, x_train, t_train, epochs=5000, lr=1e-3):
    """
    使用Adam优化器训练PINN模型。

    参数:
    pinn: PINN模型实例
    x_train: 训练数据的空间坐标(字典格式)
    t_train: 训练数据的时间坐标(字典格式)
    epochs: 训练的总轮次, 默认5000
    lr: 学习率, 默认1e-3
    """
    optimizer = torch.optim.Adam(pinn.parameters(), lr=lr)
    print("开始Adam训练...")
    for epoch in range(epochs):
        optimizer.zero_grad()
        total_loss, loss_pde, loss_ic1, loss_ic2, loss_bc = pinn.losses(x_train, t_train)
        total_loss.backward(retain_graph=True)
        optimizer.step()
        if epoch % 500 == 0:
            print(f"轮次 {epoch}: 总损失 = {total_loss.item():.6e}, "
                  f"PDE损失 = {loss_pde.item():.6e}, "
                  f"IC1损失 = {loss_ic1.item():.6e}, "
                  f"IC2损失 = {loss_ic2.item():.6e}, "
                  f"BC损失 = {loss_bc.item():.6e}")
    print("Adam训练完成。")
```

L-BFGS优化器

深度之眼
deepshare.net

```python
def train_pinn_lbfgs(pinn, x_train, t_train, max_iter=500, lr=0.1, tolerance_grad=1e-7, tolerance_change=1e-9):
    """
    使用L-BFGS优化器训练PINN模型。

    参数:
    pinn: PINN模型实例
    x_train: 训练数据的空间坐标(字典格式)
    t_train: 训练数据的时间坐标(字典格式)
    max_iter: 最大迭代次数, 默认500
    lr: 学习率, 默认0.1
    tolerance_grad: 梯度容忍度, 默认1e-7
    tolerance_change: 损失变化容忍度, 默认1e-9
    """
    optimizer = torch.optim.LBFGS(
        pinn.parameters(),
        lr=lr,
        max_iter=max_iter,
        tolerance_grad=tolerance_grad,
        tolerance_change=tolerance_change,
        line_search_fn="strong_wolfe",
        history_size=100
    )

    def closure():
        optimizer.zero_grad()
        total_loss, loss_pde, loss_ic1, loss_ic2, loss_bc = pinn.losses(x_train, t_train)
        total_loss.backward(retain_graph=True)
        return total_loss
```

前向传播与反向传播

深度之眼
deepshare.net

```python
# ## **5. 训练模型**
# 设置波速和批量大小
c_true = 2  # 真实波速
batch_size_pde = 1000  # PDE点数量
batch_size_ic = 50     # 初始条件点数量
batch_size_bc = 100    # 边界条件点数量

# 生成数据
X, T, x_test, t_test, u_test, x_train, t_train, u_train, U = get_data(
    c_true, batch_size_pde, batch_size_ic, batch_size_bc
)

# 可视化采样点
plot_sampling_points(x_train, t_train)

# 初始化模型
model = PINN(
    input_dim=2,
    output_dim=1,
    hidden_dim=100,
    num_hidden=3,
    activation='tanh'
).to(device)

# 记录训练开始时间
t1 = default_timer()

# 使用Adam优化器训练
train_pinn_adam(model, x_train, t_train, epochs=20000, lr=1e-3)

# 使用L-BFGS优化器训练
train_pinn_lbfgs(model, x_train, t_train, max_iter=10000, lr=0.1)

# 记录训练结束时间
t2 = default_timer()
print(f"训练总耗时:{t2 - t1} 秒")
```

阶段四：收敛与验证

深度之眼
deepshare.net

1. **收敛判断：** 根据损失函数的变化趋势，判断是否收敛。如果未收敛，则继续训练，调整训练策略。
2. **模型验证：** 通过测试集和实际数据对模型进行验证，评估泛化能力。

收敛判断

深度之眼
deepshare.net

```python
def plot_loss_history(pinn):
    """
    可视化训练过程中的损失历史。

    参数:
    pinn: 已训练的PINN模型实例，损失历史存储在pinn.history中
    """
    # 检查是否记录了损失历史
    if not hasattr(pinn, 'history'):
        print("No loss history available.")
        return

    epochs = range(len(pinn.history['total_loss']))

    plt.figure(figsize=(10, 6))
    plt.semilogy(epochs, pinn.history['total_loss'], label='Total Loss')
    plt.semilogy(epochs, pinn.history['loss_pde'], label='PDE Residual Loss')
    plt.semilogy(epochs, pinn.history['loss_ic1'], label='Initial Condition 1 Loss')
    plt.semilogy(epochs, pinn.history['loss_ic2'], label='Initial Condition 2 Loss')
    plt.semilogy(epochs, pinn.history['loss_bc'], label='Boundary Condition Loss')
    plt.title('Loss During Training')
    plt.xlabel('Epochs')
    plt.ylabel('Loss (Log Scale)')
    plt.legend()
    plt.grid(True)
    plt.tight_layout()
    plt.savefig("Results/loss_history.png")
    plt.show()
```

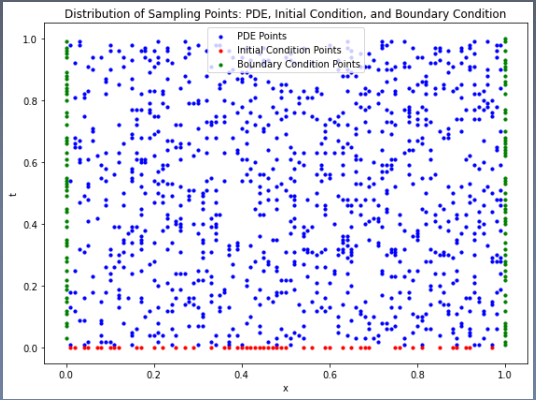
模型验证

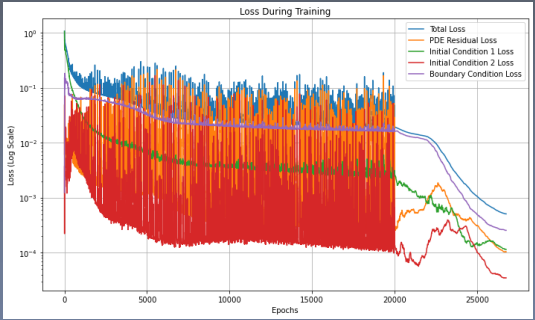
深度之眼
deepshare.net

```python
# 在训练完成后调用此函数
plot_loss_history(model)

# ## **6. 结果评估与可视化**
def relative_l2_error(pred, true):
    """计算相对L2误差"""
    return torch.norm(pred - true) / torch.norm(true)

# 计算预测结果
u_pred = model(x_test, t_test).cpu().detach().numpy()
u_error = relative_l2_error(torch.tensor(u_pred), u_test.cpu())
print(f"相对L2误差:{u_error.item()}")

# 将预测结果重塑为网格形状
u_pred_reshaped = u_pred.reshape(101, 101)

# 计算绝对误差
abs_error = np.abs(U - u_pred_reshaped)

# 创建可视化图形
fig, axs = plt.subplots(1, 3, figsize=(18, 6))

# 绘制真实解
contour_gt = axs[0].contourf(X, T, U, levels=50, cmap="viridis", vmin=np.min(U), vmax=np.max(U))
axs[0].set_title("Ground Truth")
axs[0].set_xlabel("x")
axs[0].set_ylabel("t")
plt.colorbar(contour_gt, ax=axs[0], label="True u(x, t)")

# 绘制预测解
contour_pred = axs[1].contourf(X, T, u_pred_reshaped, levels=50, cmap="viridis", vmin=np.min(U), vmax=np.max(U))
axs[1].set_title("PINN Prediction")
axs[1].set_xlabel("x")
plt.colorbar(contour_pred, ax=axs[1], label="Predicted u(x, t)")

# 绘制绝对误差
contour_error = axs[2].contourf(X, T, abs_error, levels=50, cmap="inferno", vmin=0, vmax=np.max(abs_error))
axs[2].set_title("Absolute Error")
axs[2].set_xlabel("x")
plt.colorbar(contour_error, ax=axs[2], label="Absolute Error")
```

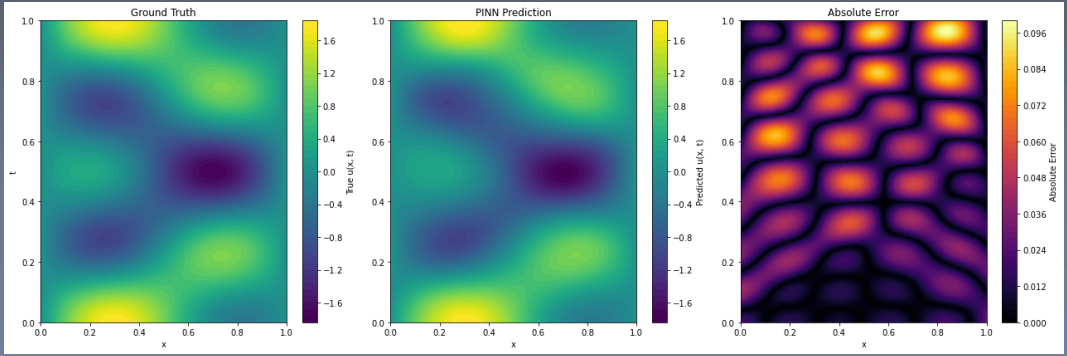
可视化：训练数据样本点与损失函数值

深度之眼
deepshare.net





可视化：预测值对比


Ground Truth
PINN Prediction
Absolute Error
t
x
True u(x, t)
Predicted u(x, t)
Absolute Error
0.0
0.2
0.4
0.6
0.8
1.0
1.6
1.2
0.8
0.4
0.0
−0.4
−0.8
−1.2
−1.6
0.096
0.084
0.072
0.060
0.048
0.036
0.024
0.012
0.000

Section 2

PINN一维波动方程逆问题求解

一维波动方程逆问题

深度之眼 deepshare.net

方程形式：

$$\frac{\partial^2 u}{\partial t^2} - c^2 \frac{\partial^2 u}{\partial x^2} = 0$$

初始条件：

$$u(x,0) = \sin(\pi x) + \sin(2\pi x), \quad \frac{\partial u(x,0)}{\partial t} = 0$$

边界条件：

$$u(0,t) = 0, \quad u(1,t) = 0$$

解析解： 根据 d'Alembert 公式，波动方程的解析解为：

$$u(x,t) = \frac{1}{2}\left[f(x+ct) + f(x-ct)\right]$$

其中 $f(x) = \sin(\pi x) + \sin(2\pi x)$。

目标：通过$c = 2$工况的观测值，使用PINN来预测c，使得预测的c与真实值相近

PINN波动方程逆问题损失函数构建

深度之眼 deepshare.net

PINN 的综合损失函数：

$$\mathcal{L}(\Theta) = \lambda_r \mathcal{L}_r(\Theta) + \lambda_{i1}\mathcal{L}_{i1}(\boldsymbol{\theta}) + \lambda_{i2}\mathcal{L}_{i2}(\boldsymbol{\theta}) + \lambda_b \mathcal{L}_b(\boldsymbol{\theta}) + \lambda_d \mathcal{L}_d(\boldsymbol{\theta})$$

各项定义为：

$$\mathcal{L}_r(\Theta) = \frac{1}{N_r}\sum_{k=1}^{N_r}\left(\frac{\partial^2 \hat{u}_\theta(\boldsymbol{x}_r^k, t_r^k)}{\partial t^2} - c^2\frac{\partial^2 \hat{u}_\theta(\boldsymbol{x}_r^k, t_r^k)}{\partial x^2}\right)^2,$$

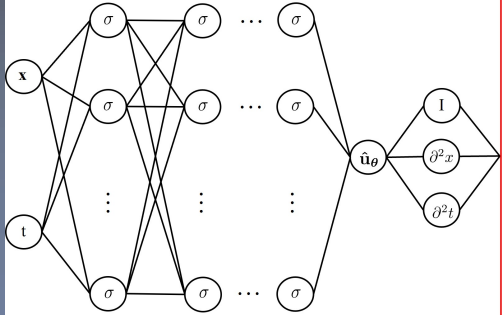
$$\mathcal{L}_{i1}(\boldsymbol{\theta}) = \frac{1}{N_i}\sum_{k=1}^{N_i}\left(\hat{u}_\theta(\boldsymbol{x}_i^k, 0) - \sin(\pi x_i^k) - \sin(2\pi x_i^k)\right)^2, \mathcal{L}_{i2}(\boldsymbol{\theta}) = \frac{1}{N_i}\sum_{k=1}^{N_i}\left(\frac{\partial \hat{u}_\theta(\boldsymbol{x}_i^k, 0)}{\partial t} - 0\right)^2$$

$$\mathcal{L}_b(\boldsymbol{\theta}) = \frac{1}{N_b}\sum_{k=1}^{N_b}\left(\hat{u}_\theta(\boldsymbol{x}_b^k, t_b^k) - 0\right)^2,$$

$$\mathcal{L}_d(\boldsymbol{\theta}) = \frac{1}{N_d}\sum_{k=1}^{N_d}\left(\hat{u}_\theta(\boldsymbol{x}_d^k, t_d^k) - u(\boldsymbol{x}_d^k, t_d^k)\right)^2.$$

PINN求解一维波动方程逆问题框架

深度之眼
deepshare.net



最小化：

$$\boldsymbol{\Theta}^\star = \arg\min_{\boldsymbol{\Theta}} [\lambda_r \mathcal{L}_r(\boldsymbol{\Theta}) + \lambda_{i1}\mathcal{L}_{i1}(\boldsymbol{\theta}) + \lambda_{i2}\mathcal{L}_{i2}(\boldsymbol{\theta}) + \lambda_b \mathcal{L}_b(\boldsymbol{\theta}) + \lambda_d \mathcal{L}_d(\boldsymbol{\theta})]$$

各项定义为：

$$\mathcal{L}_r(\boldsymbol{\Theta}) = \frac{1}{N_r}\sum_{k=1}^{N_r}\left(\frac{\partial^2 \hat{u}_{\boldsymbol{\theta}}(\boldsymbol{x}_r^k, t_r^k)}{\partial t^2} - c^2 \frac{\partial^2 \hat{u}_{\boldsymbol{\theta}}(\boldsymbol{x}_r^k, t_r^k)}{\partial x^2}\right)^2,$$

$$\mathcal{L}_{i1}(\boldsymbol{\theta}) = \frac{1}{N_i}\sum_{k=1}^{N_i}\left(\hat{u}_{\boldsymbol{\theta}}(\boldsymbol{x}_i^k, 0) - \sin(\pi x_i^k) - \sin(2\pi x_i^k)\right)^2,$$

$$\mathcal{L}_{i2}(\boldsymbol{\theta}) = \frac{1}{N_i}\sum_{k=1}^{N_i}\left(\frac{\partial \hat{u}_{\boldsymbol{\theta}}(\boldsymbol{x}_i^k, 0)}{\partial t} - 0\right)^2,$$

$$\mathcal{L}_b(\boldsymbol{\theta}) = \frac{1}{N_b}\sum_{k=1}^{N_b}\left(\hat{u}_{\boldsymbol{\theta}}(\boldsymbol{x}_b^k, t_b^k) - 0\right)^2,$$

$$\mathcal{L}_d(\boldsymbol{\theta}) = \frac{1}{N_d}\sum_{k=1}^{N_d}\left(\hat{u}_{\boldsymbol{\theta}}(\boldsymbol{x}_d^k, t_d^k) - u(\boldsymbol{x}_d^k, t_d^k)\right)^2.$$

阶段一：计算图构建

深度之眼 deepshare.net

网络架构定义： 选择神经网络结构，如全连接网络：

- **初始化逆问题物理参数**
- 输入层：空间坐标(x, t)
- 隐藏层：3层，每层100个神经元，使用tanh激活函数
- 输出层：物理场预测值$u_\theta(x, t)$

自动微分配置： 构建计算图跟踪机制，支持高阶导数计算：

- **工具：** PyTorch (‘torch.autograd‘)。
- **步骤：**
  1. 定义输入 (x, t) 并设置 ‘requires_grad=True‘。
  2. 前向传播计算 $u_\theta(x, t)$。
  3. 使用 ‘torch.autograd.grad‘ 计算 $\frac{\partial^2 u}{\partial t^2}$ 和 $\frac{\partial^2 u}{\partial x^2}$。
- **示例代码：** $u_x = grad(u_pred, x, grad_outputs = torch.ones_like(u_pred), create_graph = True)[0]$

网络架构构建

深度之眼
deepshare.net

```python
# ## **2. 定义PINNs模型**
class PINN(nn.Module):
    def __init__(self, input_dim=2, output_dim=1, hidden_dim=50, num_hidden=3, activation='tanh'):
        """
        初始化PINN模型。
        参数:
        input_dim: 输入维度, 默认2(x和t)
        output_dim: 输出维度, 默认1(u)
        hidden_dim: 隐藏层神经元数量, 默认50
        num_hidden: 隐藏层数量, 默认3
        activation: 激活函数类型, 默认'tanh', 可选'sin'
        """
        super(PINN, self).__init__()

        # 可学习参数
        self.c_hat = nn.Parameter(torch.tensor([np.random.rand()], requires_grad=True).float())

        # 定义神经网络层
        self.layers = nn.ModuleList([nn.Linear(input_dim, hidden_dim)])
        for _ in range(num_hidden - 1):
            self.layers.append(nn.Linear(hidden_dim, hidden_dim))
        self.layers.append(nn.Linear(hidden_dim, output_dim))

        # 设置激活函数
        if activation == 'tanh':
            self.activation = torch.tanh
        elif activation == 'sin':
            self.activation = torch.sin
        else:
            raise ValueError("不支持的激活函数, 仅支持 'tanh' 或 'sin'。")
```

阶段二：损失函数构建与数据准备

深度之眼 deepshare.net

1. **损失函数构建：** 根据逆问题，构建包含PDE残差、边界条件、初始条件和数据误差的损失函数：

$$\mathcal{L}(\theta) = \lambda_r \mathcal{L}_r(\theta) + \lambda_{i1} \mathcal{L}_{i1}(\theta) + \lambda_{i2} \mathcal{L}_{i2}(\theta) + \lambda_b \mathcal{L}_b(\theta) + \lambda_d \mathcal{L}_d(\theta)$$

   其中，各部分损失根据问题需求调整权重。

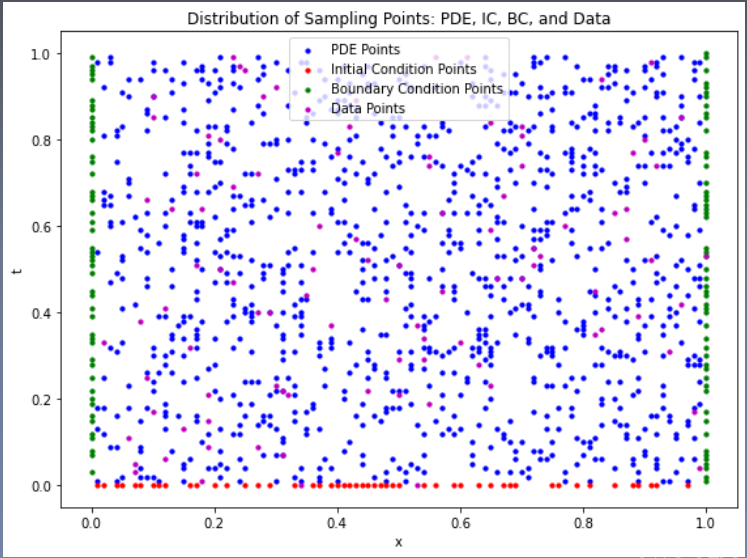
2. **训练数据生成：** 通过空间和时间采样，生成PDE解的样本点，同时准备边界条件、初始条件和观测数据的样本点：
   - 残差点$\{x_r^k\}_{k=1}^{N_r}$：均匀或随机采样
   - 边界点$\{x_b^k\}_{k=1}^{N_b}$：边界区域加密采样
   - 初始时刻点$\{x_0^k\}_{k=1}^{N_i}$：时域初始切片
   - 观测点$\{x_d^i, u_d^i\}_{i=1}^{N_d}$：均匀或随机采样

损失函数构建（PINN Class）

深度之眼
deepshare.net

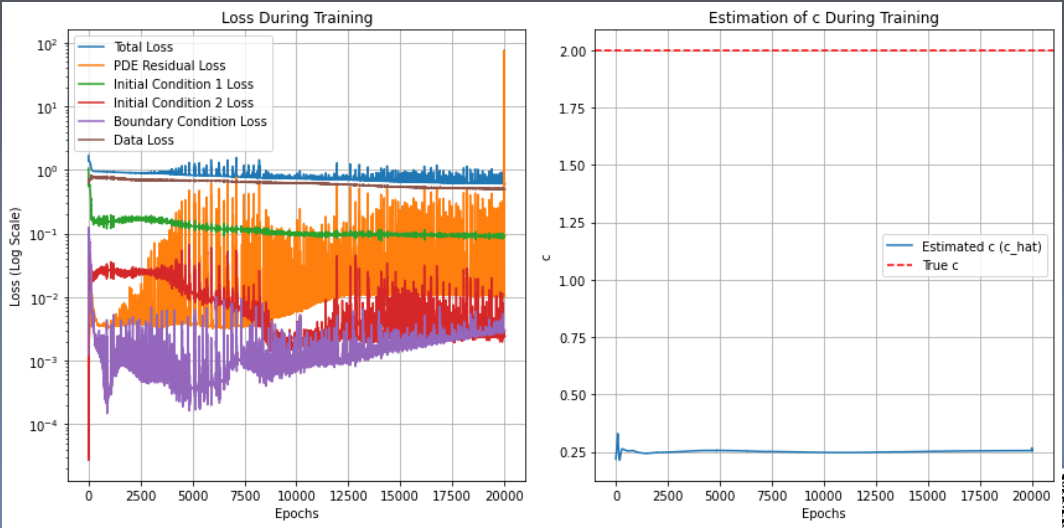
```python
    def losses(self, x, t, u_data):
        """
        计算所有损失项，包括 PDE 残差、初始条件、边界条件和数据损失。

        参数:
        x: 空间坐标(字典格式)
        t: 时间坐标(字典格式)
        u_data: 观测数据(字典格式)

        返回:
        total_loss: 总损失
        loss_pde: PDE 残差损失
        loss_ic1: 初始条件 1 损失
        loss_ic2: 初始条件 2 损失
        loss_bc: 边界条件损失
        loss_data: 数据损失
        """
        # 计算 PDE 残差
        u_pde, u_xx, u_tt = self.calculate_derivatives(x['pde'], t['pde'])
        residual = u_tt - self.c_hat**2 * u_xx
        loss_pde = torch.mean(residual**2)

        # 计算初始条件损失
        x_ic = x['ic'].requires_grad_()
        t_ic = t['ic'].requires_grad_()
        u_ic = self.forward(x_ic, t_ic)
        u_t = grad(u_ic, t_ic, grad_outputs=torch.ones_like(u_ic), create_graph=True)[0]
        loss_ic1 = torch.mean((u_ic - torch.sin(np.pi * x['ic']) - torch.sin(2 * np.pi * x['ic']))**2)
        loss_ic2 = torch.mean(u_t**2)

        # 计算边界条件损失
        loss_bc = torch.mean(self.forward(x['bc'], t['bc'])**2)

        # 计算数据损失
        u_pred_data = self.forward(x['data'], t['data'])
        loss_data = torch.mean((u_pred_data - u_data['data'])**2)

        # 总损失
        total_loss = loss_pde + loss_ic1 + loss_ic2 + loss_bc + loss_data
```

可视化：训练数据样本点

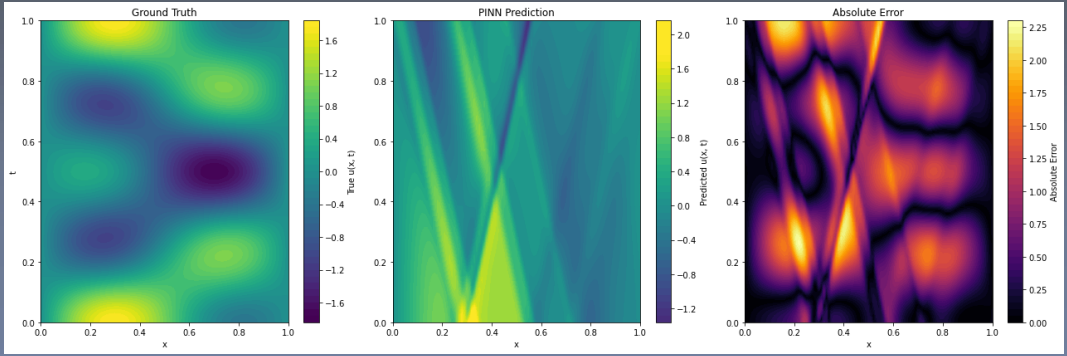

Distribution of Sampling Points: PDE, IC, BC, and Data
PDE Points
Initial Condition Points
Boundary Condition Points
Data Points
t
x
0.0
0.2
0.4
0.6
0.8
1.0

可视化：损失函数与预测波速

深度之眼
deepshare.net

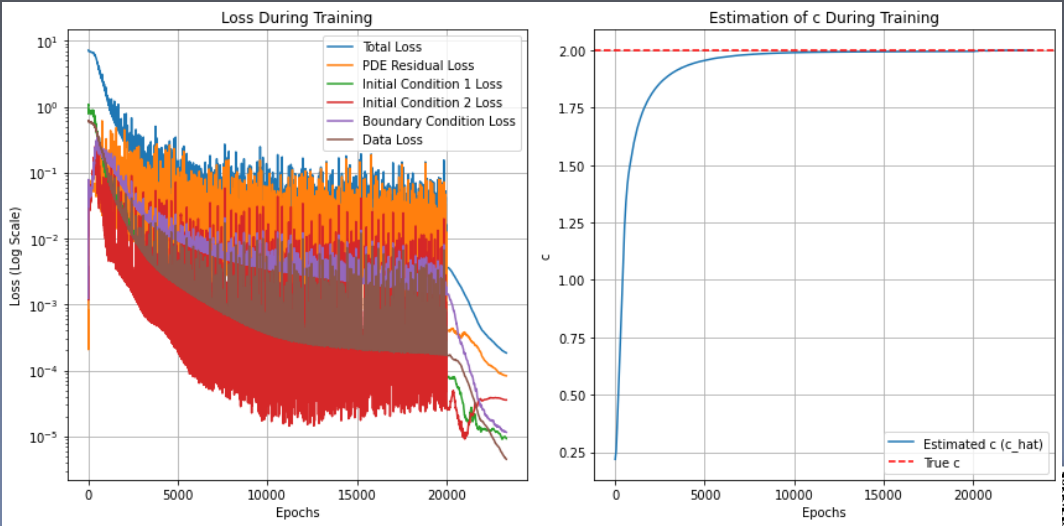


可视化：预测值对比

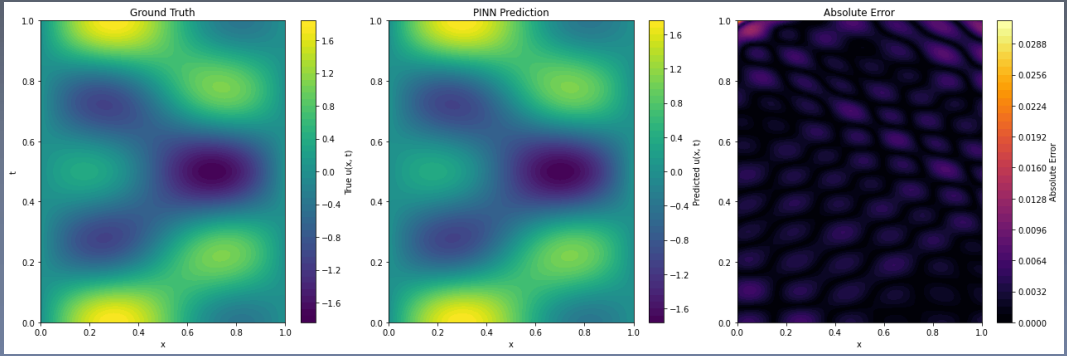

Ground Truth
PINN Prediction
Absolute Error
x
t
True u(x, t)
Predicted u(x, t)
Absolute Error

可视化：$\lambda_d = 10$损失函数与预测波速

深度之眼
deepshare.net



可视化：$\lambda_d = 10$预测值对比



作业布置

深度之眼 deepshare.net

请基于课堂内容完成以下任务：

任务 1：探索超参数对PINN正问题结果的影响（3-5个案例）

1. 修改网络结构：调整隐藏层数量（如 2, 3, 5）和每层神经元数量（如 50, 100, 200）。
2. 修改损失函数权重：尝试不同的损失函数权重（如 1, 50, 100）（导数阶数越高的loss项权重越低）。
3. 比较训练结果：记录损失曲线、收敛速度、相对 L2 误差。

任务 2：探究噪声数据对PINN逆问题模型的影响

1. 为训练数据添加不同强度的高斯噪声（如噪声标准差 0.01, 0.05, 0.1）。
2. 分析噪声对模型学习的影响：记录学习到的波速 $\hat{c}$ 和训练损失曲线。

任务 3：撰写报告

1. 总结不同超参数对PINN正问题模型训练结果的影响。
2. 分析噪声数据对PINN逆问题模型的鲁棒性，并提出改进建议。
3. 附上关键代码和图表，支持分析结果。

访问DeepSeek、ChatGPT、Claude、Grok等大模型交互平台获取辅助建议。

深度之眼
deepshare.net

Thank you